UNIVERSITAS BAKRIE

Software Quality Management

WEEK 9

AGENDA

- Quality Concept
- Review Technique
- S/W Quality Assurance

Quality Concept

UNIVERSITAS BAKRIE

Apa Itu Kualitas?

- Lima Perspektif Kualitas Menurut David Garvin
  - **Transendental**: Kualitas adalah sesuatu yang langsung dikenali tetapi sulit didefinisikan.
  - **User-based**: Kualitas tergantung pada sejauh mana produk memenuhi tujuan pengguna.
  - **Manufacturing-based**: Kualitas diukur berdasarkan kesesuaian dengan spesifikasi yang telah ditetapkan.
  - **Product-based**: Kualitas terkait dengan fitur dan karakteristik bawaan suatu produk.
  - **Value-based**: Kualitas diukur berdasarkan seberapa besar pelanggan bersedia membayar untuk produk tersebut.
- Dalam rekayasa perangkat lunak, ***quality of design*** mencerminkan sejauh mana desain memenuhi kebutuhan fungsional yang ditetapkan dalam model spesifikasi.***Quality of conformance*** menekankan pada kesesuaian implementasi dengan desain yang telah dirancang.
- Robert Glass menekankan bahwa kualitas perangkat lunak harus berorientasi pada kepuasan pengguna. Persamaan intuitifnya adalah:
  - **Kepuasan Pengguna = Produk yang Sesuai + Kualitas yang Baik + Pengiriman Tepat Waktu dan Sesuai Anggaran**

Quality Concept

Kualitas Perangkat Lunak

- Definisi umum kualitas perangkat lunak:
  - "Proses perangkat lunak yang efektif yang diterapkan dengan cara yang menciptakan produk yang berguna dan memberikan nilai yang terukur bagi pembuatnya maupun penggunanya.“
- Definisi ini menekankan pada pada tiga aspek utama:
  - Proses perangkat lunak yang efektif
  - Produk yang berguna
  - Nilai tambah bagi pengembang dan pengguna

Quality Concept

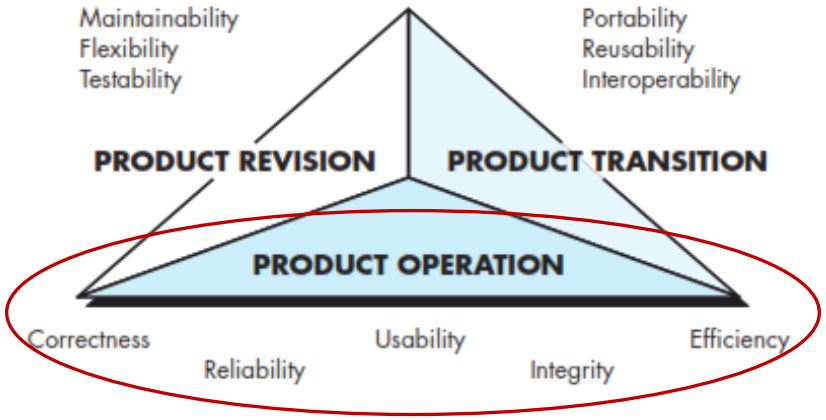
Kualitas Perangkat Lunak - Dimensi Kualitas Garvin

- David Garvin mengusulkan bahwa kualitas perangkat lunak harus dipertimbangkan dari sudut pandang delapan dimensi:
  1. **Performance Quality (Kualitas Performa),** apakah perangkat lunak menyediakan semua fitur dan fungsi yang dibutuhkan sesuai dengan model kebutuhan?
  2. **Feature Quality (Kualitas Fitur),** apakah perangkat lunak menyediakan fitur yang baik dan menyenangkan pengguna baru?
  3. **Reliability (Keandalan),** apakah perangkat lunak dapat berfungsi dengan baik tanpa kegagalan?
  4. **Conformance (Konformitas),** apakah perangkat lunak sesuai dengan standar internal dan eksternal yang relevan?
  5. **Durability (Daya Tahan),** apakah perangkat lunak dapat diperbaiki atau diperbarui tanpa menyebabkan efek samping yang tidak diinginkan?
  6. **Serviceability (Kemudahan Pemeliharaan),** apakah perangkat lunak dapat diperbaiki dalam waktu yang wajar saat terjadi kesalahan?
  7. **Aesthetics (Estetika),** perangkat lunak yang estetis memiliki desain yang elegan, alur kerja yang alami, dan tampilan yang menarik..
  8. **Perception (Persepsi Pengguna),** persepsi kualitas perangkat lunak sering kali dipengaruhi oleh pengalaman sebelumnya atau reputasi pengembang.

Quality Concept

Kualitas Perangkat Lunak – Faktor-Faktor Kualitas McCall

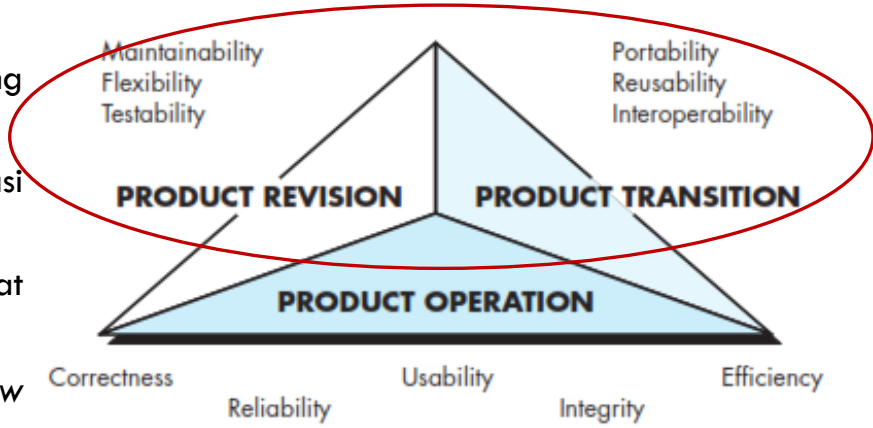
- McCall, Richards, dan Walters (1977) mengusulkan kategori faktor-faktor kualitas perangkat lunak yang berfokus pada tiga aspek utama:
  - **Karakteristik operasional (*Operational characteristics*)**
    1. **Correctness (Kebenaran)**, sejauh mana perangkat lunak memenuhi spesifikasi dan tujuan bisnis pengguna.
    2. **Reliability (Keandalan)**, seberapa baik perangkat lunak berfungsi tanpa kegagalan dalam kondisi operasional yang diharapkan.
    3. **Efficiency (Efisiensi)**, seberapa optimal perangkat lunak dalam menggunakan sumber daya komputasi, seperti CPU, memori, dan bandwidth jaringan.
    4. **Integrity (Integritas)**, sejauh mana perangkat lunak dapat melindungi data dan akses dari pengguna yang tidak sah.
    5. **Usability (Kemudahan Penggunaan)**, seberapa mudah pengguna dapat belajar dan menggunakan perangkat lunak.



Quality Concept

Kualitas Perangkat Lunak – Faktor-Faktor Kualitas McCall (2)

- McCall, Richards, dan Walters (1977) mengusulkan kategori faktor-faktor kualitas perangkat lunak yang berfokus pada tiga aspek utama:
  - Kemampuan untuk mengalami perubahan (*Ability to undergo change*)
    - **Maintainability (Kemudahan Pemeliharaan)**, seberapa mudah pengembang dapat menemukan dan memperbaiki kesalahan dalam perangkat lunak.
    - **Flexibility (Fleksibilitas)**, seberapa mudah perangkat lunak dapat dimodifikasi untuk memenuhi kebutuhan baru.
    - **Testability (Kemudahan Pengujian)**, seberapa mudah perangkat lunak dapat diuji untuk memastikan bahwa semua fitur berfungsi dengan benar.
  - Kemampuan beradaptasi ke lingkungan baru (*Adaptability to new environments*)
    - **Portability (Portabilitas)**, seberapa mudah perangkat lunak dapat dipindahkan dari satu lingkungan ke lingkungan lain.
    - **Reusability (Dapat Digunakan Kembali)**, seberapa banyak bagian dari perangkat lunak dapat digunakan kembali dalam aplikasi lain.
    - **Interoperability (Interoperabilitas)**, seberapa mudah perangkat lunak dapat bekerja bersama dengan sistem lain.



Quality Concept

Kualitas Perangkat Lunak – Faktor-Faktor Kualitas ISO 9126

- (1) **Functionality (Fungsionalitas),** Sejauh mana perangkat lunak memenuhi kebutuhan pengguna berdasarkan lima subatribut: Suitability (Kesesuaian), Accuracy (Akurasi), Interoperability (Interoperabilitas), Compliance (Kepatuhan), dan Security (Keamanan).
- (2) **Reliability (Keandalan),** seberapa lama perangkat lunak dapat bekerja tanpa mengalami kegagalan, berdasarkan tiga subatribut: Maturity (Kematangan), Fault Tolerance (Ketahanan terhadap Kesalahan), dan Recoverability (Kemampuan Pulih).
- (3) **Usability (Kemudahan Penggunaan),** Seberapa mudah pengguna dapat menggunakan perangkat lunak, berdasarkan tiga subatribut: Understandability (Dapat Dipahami), Learnability (Mudah Dipelajari), Operability (Dapat Dioperasikan dengan Mudah).
- (4) **Efficiency (Efisiensi),** seberapa optimal perangkat lunak dalam menggunakan sumber daya sistem, berdasarkan dua subatribut: Time Behavior (Performa Waktu) dan Resource Behavior (Pemanfaatan Sumber Daya).
- (5) **Maintainability (Kemudahan Pemeliharaan),** seberapa mudah perangkat lunak dapat diperbaiki dan diperbarui, berdasarkan empat subatribut: Analyzability (Kemudahan Analisis), Changeability (Kemudahan Perubahan), Stability (Stabilitas), dan Testability (Kemudahan Pengujian).
- (6) **Portability (Portabilitas),** seberapa mudah perangkat lunak dapat dijalankan di lingkungan yang berbeda, berdasarkan empat subatribut: Adaptability (Dapat Beradaptasi), Installability (Kemudahan Instalasi), Conformance (Kepatuhan terhadap Standar), dan Replaceability (Kemudahan Penggantian).

Quality Concept

UNIVERSITAS BAKRIE

Kualitas Perangkat Lunak – Targeted Quality Factors

- Dalam pengembangan perangkat lunak yang lebih spesifik, sebuah tim pengembang perlu menetapkan karakteristik kualitas yang ditargetkan dan mengajukan pertanyaan spesifik untuk menilai sejauh mana faktor kualitas telah dipenuhi.
- Tujuan utama dari ***Targeted Quality Factors***:
  - Menyediakan metode evaluasi yang lebih terfokus pada aspek spesifik perangkat lunak.
  - Memungkinkan pengujian kualitas yang lebih mendetail, terutama dalam elemen antarmuka pengguna (UI).
  - Menggunakan parameter yang dapat diukur untuk menilai aspek kualitas tertentu.

Quality Concept

Kualitas Perangkat Lunak – Targeted Quality Factors(2)

- Contoh: Evaluasi Faktor Kualitas "Usability“ McCall mengidentifikasi usability (kemudahan penggunaan) sebagai faktor kualitas penting. Untuk menilai usability, kita perlu mengevaluasi atribut yang lebih spesifik, seperti:
  - Intuitiveness (Keintuitifan),
    - Apakah tata letak antarmuka mudah dipahami?
    - Apakah operasi di dalam antarmuka mudah ditemukan dan diakses?
  - Efficiency (Efisiensi)
    - Apakah tata letak antarmuka memungkinkan pengguna menemukan fitur dengan cepat?
    - Apakah urutan operasi atau input data dapat dilakukan dengan gerakan yang efisien?
  - Robustness (Ketahanan terhadap Kesalahan)
    - Apakah perangkat lunak mengenali data yang salah dan memberikan peringatan yang sesuai?
    - Apakah perangkat lunak dapat tetap berjalan meskipun terjadi kesalahan input?
  - Richness (Kekayaan Fitur)
    - Apakah antarmuka dapat dikustomisasi sesuai dengan kebutuhan pengguna?
    - Apakah ada fitur makro yang memungkinkan pengguna melakukan serangkaian operasi dengan satu perintah?

Quality Concept

UNIVERSITAS BAKRIE

Dilema Kualitas Perangkat Lunak

- Bertrand Meyer dalam wawancara mengemukakan "Dilema Kualitas Perangkat Lunak":
  - Jika kualitas perangkat lunak buruk, pengguna tidak akan membeli atau menggunakan produk tersebut.
  - Jika terlalu banyak waktu, tenaga, dan biaya dihabiskan untuk menciptakan perangkat lunak yang sempurna, maka produk akan terlambat masuk pasar atau biaya produksi menjadi terlalu mahal sehingga perusahaan bisa bangkrut.
  - Industri perangkat lunak berusaha mencapai keseimbangan antara kualitas dan kecepatan rilis, agar produk cukup baik untuk diterima pengguna tetapi tetap ekonomis.
- Kenyataan ini menjadi tantangan besar bagi perusahaan pengembang perangkat lunak, di mana mereka harus memilih antara kesempurnaan atau keberlanjutan bisnis.

Quality Concept

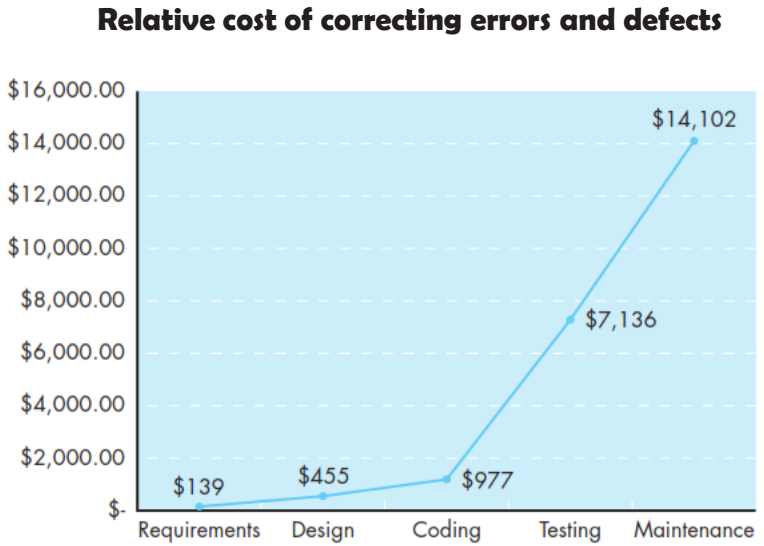
Dilema Kualitas Perangkat Lunak - Konsep "Good Enough" Software

- Apa yang dimaksud dengan "good enough"?
  - Perangkat lunak memberikan fitur utama yang berkualitas tinggi kepada pengguna.
  - Terdapat beberapa bug pada fitur yang kurang penting atau jarang digunakan.
  - Pengembang berharap pengguna dapat mentoleransi bug kecil karena fitur utama berjalan dengan baik.
- Dapat Berhasil bagi Perusahaan Besar, perusahaan besar dengan anggaran pemasaran besar dapat mempengaruhi pasar dan "mengunci" pengguna sejak versi 1.0, mereka dapat berjanji untuk memperbaiki bug di versi berikutnya.Strategi ini bisa berhasil jika basis pengguna sudah besar dan setia.
- Risiko bagi Perusahaan Kecil, perusahaan kecil tidak memiliki sumber daya untuk menghadapi dampak negatif dari perangkat lunak berkualitas rendah.Jika reputasi buruk menyebar, produk tidak akan mendapatkan kesempatan kedua.Ulasan negatif dan kehilangan kepercayaan dapat membuat perusahaan kecil bangkrut sebelum versi 2.0 dirilis.
- Risiko dalam Domain Aplikasi Tertentu, dalam perangkat lunak real-time, sistem tertanam, atau aplikasi kritis seperti avionik, otomotif, atau telekomunikasi, bug dapat menyebabkan kecelakaan atau kegagalan fatal. Perusahaan dalam industri ini tidak boleh mengandalkan konsep "good enough" karena risiko hukum dan keamanannya tinggi.

Quality Concept

Dilema Kualitas Perangkat Lunak - Biaya Kualitas (The Cost of Quality)

- Banyak orang berpikir bahwa mengejar kualitas tinggi membutuhkan terlalu banyak waktu dan uang. Namun, kualitas yang buruk juga membawa biaya, baik untuk pengguna (yang harus menghadapi bug), maupun organisasi pengembang (yang harus memperbaikinya).
- Biaya kualitas mencakup semua biaya yang berkaitan dengan biaya mengejar kualitas (aktivitas QA/QC) dan dampak dari kualitas rendah.
- Biaya kualitas terdiri dari dari tiga biaya yang berkaitan dengan:
  - Biaya Pencegahan (Prevention Costs) → Biaya perencanaan pengujian
  - Biaya Penilaian (Appraisal Costs) → Biaya pengujian dan debugging
  - Biaya Kegagalan (Failure Costs)
    - Internal Failure: Kesalahan ditemukan sebelum rilis.
    - External Failure: Kesalahan ditemukan setelah rilis.



Mana yang lebih besar, **biaya untuk mencapai kualitas**, atau **biaya akibat kualitas rendah**?

Quality Concept

Dilema Kualitas Perangkat Lunak - Risiko (Risks)

- Implikasi utama: Jika kualitas perangkat lunak buruk, maka risiko meningkat, baik bagi:
  - Pengembang (developer)
  - Pengguna akhir (end user)
- Kualitas rendah = Risiko tinggi.
- Contoh Nyata – Tragedi Panama (2000) di sebuah rumah sakit di Panama, selama November 2000: 28 pasien menerima overdosis sinar gamma untuk terapi kanker. Akibatnya:5 orang meninggal karena keracunan radiasi.15 lainnya mengalami komplikasi serius. Penyebabnya: Perangkat lunak buatan perusahaan AS dimodifikasi oleh teknisi rumah sakit.Modifikasi itu mengubah perhitungan dosis radiasi—dan tidak diuji dengan baik. Konsekuensi: 3 fisikawan medis di Panama didakwa dengan pembunuhan tingkat dua. Perusahaan pengembang perangkat lunak dituntut di dua negara.
- Risikonya bukan hanya soal biaya, tapi bisa berupa:
  - Hukum
  - Kehilangan nyawa
  - Kehilangan reputasi
  - Kehancuran bisnis
- Pengembang harus sadar bahwa mereka bertanggung jawab atas kode yang mereka hasilkan.

Quality Concept

UNIVERSITAS BAKRIE

Dilema Kualitas Perangkat Lunak -Kualitas dan Keamanan

- Perangkat lunak berkualitas rendah → lebih rentan terhadap serangan → menambah risiko keamanan.
- Implikasi: Kualitas dan keamanan tidak dapat dipisahkan, karena kualitas rendah = potensi celah keamanan.
- Gary McGraw menekankan bahwa keamanan adalah bagian integral dari kualitas perangkat lunak. Keamanan harus dipikirkan sejak awal: desain, arsitektur, pengujian, dan pengkodean.
- Banyak organisasi hanya fokus pada **bug** (kesalahan implementasi), padahal **flaws** (cacat arsitektural) justru lebih berbahaya dan sering diabaikan.

Quality Concept

Dilema Kualitas Perangkat Lunak - Dampak Tindakan Manajerial terhadap Kualitas Perangkat Lunak

- Kualitas perangkat lunak tidak hanya dipengaruhi oleh teknologi dan teknik rekayasa perangkat lunak, tetapi juga oleh keputusan manajemen proyek. Bahkan metode terbaik bisa gagal jika keputusan manajemen salah arah.
- Keputusan Estimasi, Tim pengembang sering harus menyesuaikan estimasi dengan jadwal dan anggaran yang sudah ditentukan dari awal.Namun tekanan time-to-market menyebabkan pengambilan jalan pintas—uji coba dipersingkat, dokumentasi minim, analisis tidak mendalam.
- Keputusan Penjadwalan, pengujian komponen dilakukan secara berurutan sesuai dependensi.namun jika waktu mepet, tim mungkin menguji komponen sebelum semua dependensinya siap → potensi cacat tersembunyi meningkat.
- Keputusan Berorientasi Risiko, banyak tim tidak menyiapkan rencana kontingensi karena yakin semuanya akan berjalan lancar. Namun ketika risiko menjadi kenyataan (misalnya anggota tim sakit, modul gagal sinkron), proyek panik sehingga kualitas jatuh drastis.

Quality Concept

UNIVERSITAS BAKRIE

Mencapai Kualitas Perangkat Lunak

- Manajemen proyek yang baik dan praktik rekayasa perangkat lunak yang solid diterapkan dalam empat kegiatan utama untuk mencapai kualitas perangkat lunak yang tinggi:
  - Metode rekayasa perangkat lunak, menggunakan konsep dan metode analisis yang benar dapat meningkatkan kemungkinan penciptaan perangkat lunak berkualitas tinggi.
  - Teknik manajemen proyek, Pengelolaan proyek yang baik, dengan estimasi yang realistis, pemahaman jadwal, dan perencanaan risiko yang matang, akan berdampak positif pada kualitas perangkat lunak.
  - Tindakan pengendalian kualitas, pengendalian kualitas memastikan bahwa setiap produk yang dihasilkan memenuhi standar yang telah ditetapkan, dengan melakukan inspeksi dan pengujian yang berkelanjutan.
  - Jaminan kualitas perangkat lunak, Jaminan kualitas memastikan bahwa pengendalian kualitas berjalan dengan baik dan memberikan data yang dibutuhkan untuk membuat keputusan perbaikan jika diperlukan.

Quality Concept

Evaluasi dan Diskusi

1. Apa yang dimaksud dengan perspektif kualitas **'User-based'**?
   A) Kualitas diukur berdasarkan kesesuaian produk dengan spesifikasi yang ditetapkan.
   B) Kualitas diukur berdasarkan seberapa baik produk memenuhi kebutuhan pengguna.
   C) Kualitas dilihat dari aspek estetika dan desain produk.
   D) Kualitas diukur berdasarkan nilai yang dibayar oleh konsumen.

Quality Concept

Evaluasi dan Diskusi

1. Apa yang dimaksud dengan perspektif kualitas **'User-based'**?
   A) Kualitas diukur berdasarkan kesesuaian produk dengan spesifikasi yang ditetapkan.
   B) Kualitas diukur berdasarkan seberapa baik produk memenuhi kebutuhan pengguna.
   C) Kualitas dilihat dari aspek estetika dan desain produk.
   D) Kualitas diukur berdasarkan nilai yang dibayar oleh konsumen.

Quality Concept

Evaluasi dan Diskusi

2. Apa yang dimaksud dengan **'good enough'** dalam konteks kualitas perangkat lunak?

A) Perangkat lunak yang hampir sempurna, tanpa cacat sama sekali.

B) Perangkat lunak yang tidak membutuhkan pengujian sama sekali.

C) Perangkat lunak yang memenuhi fitur utama meskipun memiliki beberapa bug kecil di fitur minor.

D) Perangkat lunak yang hanya dapat berfungsi pada satu platform.

Quality Concept

Evaluasi dan Diskusi

3. Jika kualitas perangkat lunak buruk, apa yang akan terjadi pada risikonya?.

A) Risiko menurun karena pengguna lebih toleran.

B) Risiko hanya berlaku untuk pengembang perangkat lunak.

C) Risiko hanya berdampak pada sistem operasi.

D) Risiko meningkat baik bagi pengembang maupun pengguna.

Quality Concept

Evaluasi dan Diskusi

4. Jelaskan bagaimana sebuah tim pengembang perangkat lunak dapat menjaga keseimbangan antara kualitas perangkat lunak dan keterbatasan waktu serta anggaran yang ada. Berikan contoh risiko yang bisa terjadi jika kualitas perangkat lunak tidak diprioritaskan.

Review Technique

Dampak Biaya dari Cacat Perangkat Lunak (Cost Impact of Software Defects)

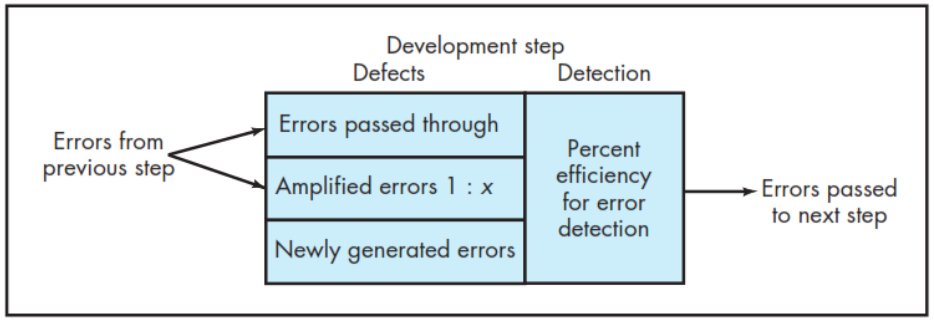
- **Tinjauan teknis (reviews)** berfungsi sebagai "filter" dalam proses rekayasa perangkat lunak untuk menemukan kesalahan (errors) dan cacat (defects) sebelum produk sampai ke pengguna.
- Tujuan utama: meningkatkan kualitas produk dengan menemukan kesalahan sedini mungkin.
- **Error vs. Defect:**
  - **Error:** Masalah kualitas yang ditemukan sebelum rilis (misalnya, kesalahan logika dalam kode).
  - **Defect:** Masalah kualitas yang ditemukan setelah rilis (misalnya, bug yang dilaporkan pengguna).
- Biaya perbaikan defect jauh lebih tinggi daripada error.

Review Technique

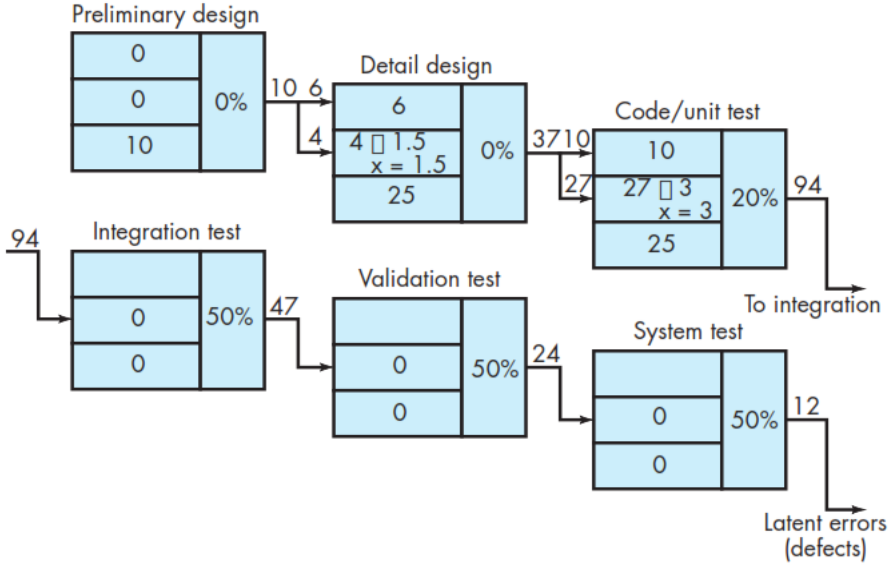
Amplifikasi dan Penghapusan Cacat (Defect Amplification and Removal)

- Kesalahan dalam tahap awal (misalnya, desain) dapat "teramplifikasi" menjadi lebih banyak kesalahan di tahap selanjutnya (misalnya, kode).
- Model amplifikasi cacat menunjukkan bagaimana tinjauan mengurangi jumlah kesalahan yang lolos ke tahap berikutnya.

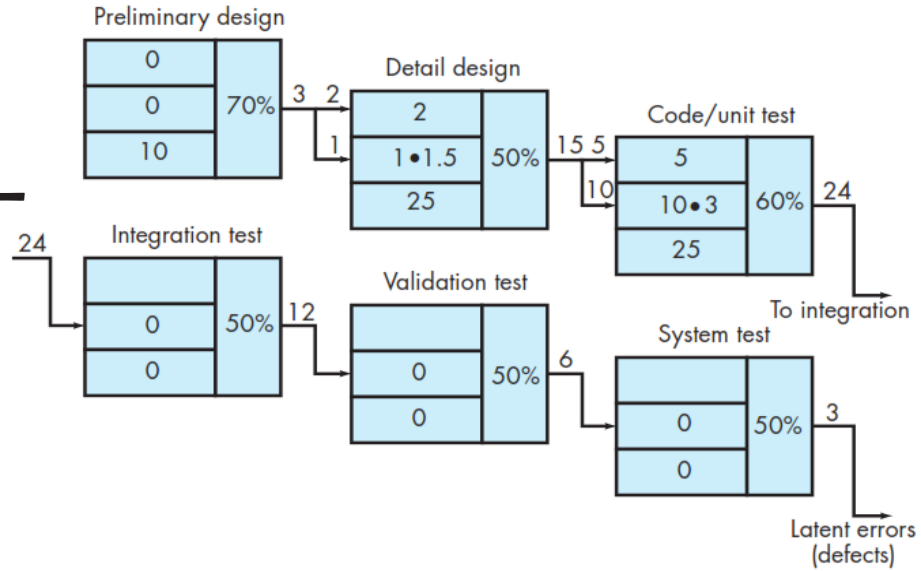
Diagram amplifikasi cacat



Defect amplification — no reviews



Defect amplification— reviews conducted



Review Technique

Metrik Tinjauan dan Penggunaannya (Review Metrics and Their Use)

- Metrik tinjauan membantu mengukur efektivitas proses tinjauan:
  - Upaya Persiapan (*Preparation effort*)-Ep, merupakan upaya yang diperlukan (dalam satuan jam orang) untuk meninjau produk kerja sebelum pertemuan tinjauan yang sesungguhnya.
  - Upaya Penilaian (*Assessment effort*)-Ea, merupakan upaya yang dikeluarkan (dalam jam orang) selama tinjauan aktual.
  - Upaya Perbaikan (*Rework effort*)-Er, merupakan upaya (dalam jam orang) yang didedikasikan untuk memperbaiki kesalahan yang ditemukan selama tinjauan.
  - Ukuran Produk Kerja *(Work product size)-WPS*, merupakan ukuran ukuran produk kerja yang telah ditinjau (misalnya, jumlah model UML, jumlah halaman dokumen, atau jumlah baris kode).
  - Kesalahan Kecil (*Minor errors found*)-(Err_{minor}), jumlah kesalahan yang ditemukan dan dikategorikan sebagai kecil (memerlukan lebih sedikit upaya untuk diperbaiki).
  - Kesalahan Besar (*Major errors found*),-($Err_{minor)}$, iumlah kesalahan yang ditemukan dan dikategorikan sebagai besar (memerlukan lebih banyak upaya untuk diperbaiki).

Review Technique

Metrik Tinjauan dan Penggunaannya (Review Metrics and Their Use) (2)

- Upaya Tinjauan (E_review): $\mathbf{E_p + E_a + E_r}$
- Total Kesalahan (Err_{tot}) = $\mathbf{Err_{minor} + Err_{major}}$
- Kepadatan Kesalahan (*Error Density*): kepadatan kesalahan mengukur jumlah kesalahan yang ditemukan per unit produk kerja yang ditinjau. $\mathbf{Err_{tot}}$ / **WPS**
- Contoh:Misalkan model kebutuhan memiliki 18 diagram UML dari 32 halaman materi deskriptif. Tinjuan menemukan 18 kesalahan minor dan 4 kesalahan besar. Maka:
  - $Err_{tot} = 18 + 4 = 22$ kesalahan.
  - Error density = 22 / 18 = 1.2 kesalahan per diagram UML atau 22 / 32 = 0.68 kesalahan per halaman model kebutuhan.

$$E_{review} = E_p + E_a + E_r$$

$$Err_{tot} = Err_{minor} + Err_{major}$$

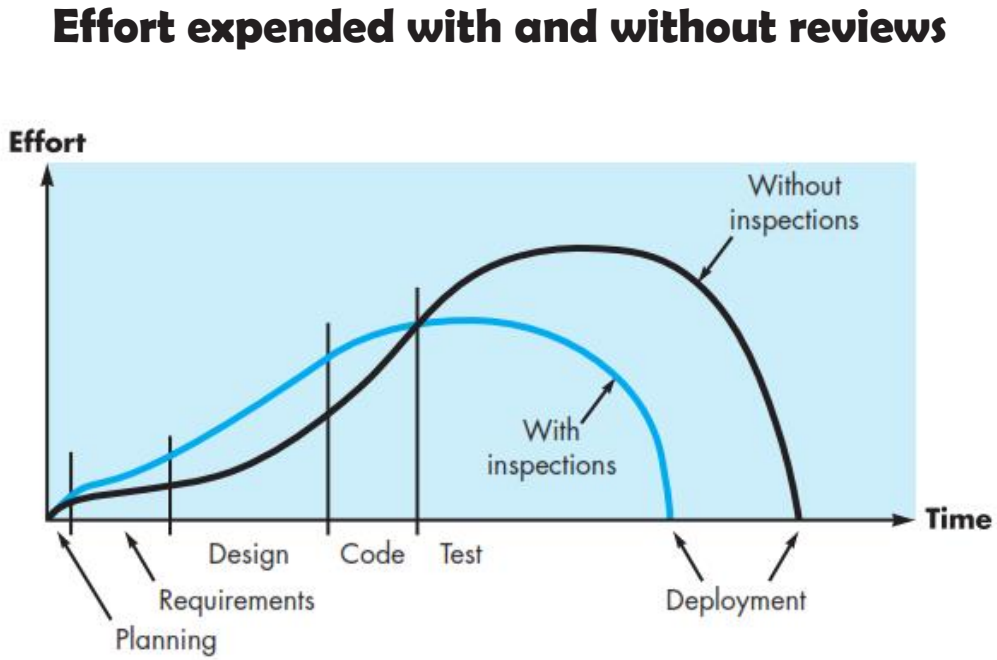
Error density represents the errors found per unit of work product reviewed.

$$\text{Error density} = \frac{Err_{tot}}{WPS}$$

Review Technique

Efektivitas Biaya Tinjauan (Cost-Effectiveness of Reviews)

- Mengukur efektivitas biaya dari tinjauan teknis secara langsung sangat sulit. Organisasi rekayasa perangkat lunak hanya dapat menilai efektivitas dan keuntungan biaya tinjauan setelah tinjauan selesai, metrik tinjauan terkumpul, data rata-rata dihitung, dan kualitas perangkat lunak diukur melalui pengujian.
- Penghematan waktu ini dapat mengarah pada siklus pengiriman yang lebih pendek dan peningkatan waktu untuk pemasaran. Artinya, dengan mengidentifikasi dan memperbaiki kesalahan lebih awal melalui tinjauan, proyek perangkat lunak dapat diselesaikan lebih cepat dan dengan kualitas yang lebih baik.



Review Technique

Evaluasi dan Diskusi

5. Perbedaan utama antara 'error' dan 'defect' adalah:

A) Error ditemukan sebelum rilis, defect ditemukan setelah rilis.

B) Error ditemukan setelah rilis, defect ditemukan sebelum rilis.

C) Error adalah kesalahan minor, defect adalah kesalahan besar.

D) Error ditemukan selama pengujian, defect ditemukan selama tinjauan.

Review Technique

Evaluasi dan Diskusi

6. Perbedaan utama antara 'error' dan 'defect' adalah:

A) Error ditemukan sebelum rilis, defect ditemukan setelah rilis.

B) Error ditemukan setelah rilis, defect ditemukan sebelum rilis.

C) Error adalah kesalahan minor, defect adalah kesalahan besar.

D) Error ditemukan selama pengujian, defect ditemukan selama tinjauan.

Review Technique

Evaluasi dan Diskusi

7. Hitung error density dari model kebutuhan memiliki 40 diagram UML dari 70 halaman materi deskriptif. Tinjuan menemukan 43 kesalahan minor dan 15 kesalahan besar.

S/W Quality Assurance

UNIVERSITAS BAKRIE

Latar Belakang Masalah

- Quality Control (QC) dan Quality Assurance (QA) adalah aktivitas penting bagi semua perusahaan yang memproduksi barang/jasa untuk digunakan oleh pihak lain.
- **QA perangkat lunak** adalah:"Pola tindakan yang direncanakan dan sistematis yang dibutuhkan untuk memastikan kualitas perangkat lunak yang tinggi." (Schneidewind, 1998)
- A bukan hanya tugas satu orang atau tim. Semua pihak terlibat, antara lain: Software engineer (pengembang), Manajer proyek, Customer, Salesperson, Tim SQA (Software Quality Assurance) khusus.
- QA adalah tanggung jawab kolektif, bukan hanya bagian akhir dari proyek. QA dimulai dari tahap perencanaan, berjalan sepanjang proses pengembangan, dan melibatkan semua orang dalam organisasi.

S/W Quality Assurance

UNIVERSITAS BAKRIE

Elemen-Elemen Jaminan Kualitas S/W

- 10 Elemen Utama dalam SQA:
  - Standar, sumber: IEEE, ISO, dan lembaga standar lainnya, penerapannya bisa sukarela (dipilih oleh organisasi) atau wajib (dari pelanggan/pemerintah). Tugas SQA: memastikan standar ini diikuti dan semua produk kerja sesuai standar.
  - Review dan Audit, review teknis adalah kontrol kualitas oleh sesama engineer untuk mencari error. Audit adalah pemeriksaan oleh tim SQA untuk memastikan prosesnya dilakukan dengan baik dan benar.
  - Pengujian (Testing) untuk menemukan kesalahan (errors). SQA tidak melakukan pengujian langsung, tapi memastikan perencanaan dan pelaksanaan testing efisien.
  - Pengumpulan dan Analisis Cacat/Error untuk memperbaiki proses dengan belajar dari data error. Data ini digunakan untuk menganalisis pola kesalahan, lalu menentukan aktivitas yang paling efektif untuk mengurangi error.
  - Manajemen Perubahan, perubahan adalah bagian alami dari proyek perangkat lunak.Jika tidak dikontrol, perubahan bisa menyebabkan kekacauan dan menurunnya kualitas. SQA memastikan perubahan dikelola dengan proses yang tepat.
  - Edukasi, pada peningkatan berkelanjutan hanya bisa terjadi jika semua pihak mendapatkan edukasi tentang teknik, alat, dan praktik terbaik.SQA berperan aktif dalam pelatihan dan penyebaran pengetahuan.

S/W Quality Assurance

UNIVERSITAS BAKRIE

Elemen-Elemen Jaminan Kualitas S/W (2)

- 10 Elemen Utama dalam SQA:
  - Manajemen Vendor, SQA memastikan vendor mengikuti praktik kualitas, bahkan bisa dimasukkan ke dalam kontrak.
  - Manajemen Keamanan (Security), SQA harus memastikan bahwa proses pengembangan mengintegrasikan prinsip keamanan.
  - Keselamatan (Safety), jika software gagal, bisa membahayakan nyawa manusia (contoh: otomotif, medis).SQA bisa menilai risiko dan memastikan langkah-langkah mitigasi diterapkan.
  - Manajemen Risiko, Engineer memang bertugas menganalisis dan mengurangi risiko.Tapi SQA harus memastikan kegiatan manajemen risiko dilakukan dan rencana darurat disiapkan.

S/W Quality Assurance

UNIVERSITAS BAKRIE

Proses SQA dan Karakteristik Produk

- Setiap produk perangkat lunak punya kebutuhan kualitas yang berbeda. Oleh karena itu, proses dan aktivitas SQA juga harus disesuaikan dengan karakteristik produk tersebut.
- Untuk menjamin kualitas yang sesuai, tim SQA harus memahami karakteristik kualitas yang diinginkan (misalnya: keandalan, efisiensi, keamanan). Memilih proses dan aktivitas SQA yang relevan untuk mencapainya.
- Dua framework SQA yang sering digunakan:
  - CMMI (Capability Maturity Model Integration) – dari SEI (Software Engineering Institute)
    - Fokus: kematangan proses organisasi.
    - Tujuan: memperbaiki proses secara bertahap → kualitas produk ikut meningkat.
  - ISO 9000 – Standar kualitas internasional, Fokus: manajemen kualitas secara umum, tidak terbatas pada software. Cocok untuk organisasi yang ingin memenuhi standar global dalam pengelolaan mutu.

S/W Quality Assurance

UNIVERSITAS BAKRIE

Tugas-Tugas SQA

- Tim SQA tidak hanya mengawasi, tapi juga membantu tim proyek agar produk akhir benar-benar berkualitas tinggi. Menurut SEI (Software Engineering Institute), tugas utama tim SQA mencakup:
  - Membuat Rencana SQA (SQA Plan)
  - Terlibat dalam Deskripsi Proses Software
  - Meninjau Aktivitas Rekayasa Software
  - Audit Produk Kerja Software
  - Pastikan Penyimpangan Ditangani Secara Formal
  - Catat dan Laporkan Ketidaksesuaian ke Manajemen
  - Koordinasi Manajemen Perubahan

S/W Quality Assurance

Tujuan, Atribut, dan Metrik SQA

- Setiap tujuan punya atribut tertentu yang bisa diukur menggunakan metrik, agar kita bisa tahu apakah tujuan tersebut tercapai atau belum.Berikut ini adalah 4 tujuan utama:
  - Kualitas Kebutuhan (Requirements Quality)
  - Kualitas Desain (Design Quality)
  - Kualitas Kode (Code Quality)
  - Efektivitas Pengendalian Kualitas (Quality Control Effectiveness)

Tujuan Utama	Fokus Atribut	Contoh Metrik	Contoh SafeHome
Kualitas Kebutuhan	Benar, lengkap, konsisten	Jumlah requirement ambigu	Kebutuhan SMS alert
Kualitas Desain	Modularitas, kesesuaian	Coupling antar modul	Arsitektur sensor
Kualitas Kode	Readability, complexity	Cyclomatic complexity	Modul login
Efektivitas QC	Deteksi & pencegahan bug	Bug ditemukan/test	Unit testing efisien

Goal	Attribute	Metric
Requirement quality	Ambiguity	Number of ambiguous modifiers (e.g., many, large, human-friendly)
	Completeness	Number of TBA, TBD
	Understandability	Number of sections/subsections
	Volatility	Number of changes per requirement
		Time (by activity) when change is requested
	Traceability	Number of requirements not traceable to design/code
	Model clarity	Number of UML models
		Number of descriptive pages per model
		Number of UML errors
Design quality	Architectural integrity	Existence of architectural model
	Component completeness	Number of components that trace to architectural model
		Complexity of procedural design
	Interface complexity	Average number of pick to get to a typical function or conte
		Layout appropriateness
	Patterns	Number of patterns used
Code quality	Complexity	Cyclomatic complexity
	Maintainability	Design factors (Chapter 8)
	Understandability	Percent internal comments
		Variable naming conventions
	Reusability	Percent reused components
	Documentation	Readability index
QC effectiveness	Resource allocation	Staff hour percentage per activity
	Completion rate	Actual vs. budgeted completion time
	Review effectiveness	See review metrics (Chapter 14)
	Testing effectiveness	Number of errors found and criticality
		Effort required to correct an error
		Origin of error

S/W Quality Assurance

UNIVERSITAS BAKRIE

Jaminan Kualitas Perangkat Lunak Secara Statistik (Statistical SQA)

- **Statistical SQA** adalah pendekatan kuantitatif terhadap kualitas perangkat lunak, di mana data kesalahan (error) dan cacat (defect) dianalisis untuk mengidentifikasi akar masalah dan melakukan perbaikan pada proses yang menyebabkannya.
- Langkah-Langkah Statistical SQA
  - Kumpulkan dan kategorikan data kesalahan
  - Telusuri akar penyebabnya
  - Gunakan Prinsip Pareto (80/20), 80% dari semua kesalahan biasanya bisa ditelusuri ke 20% penyebab utama.
  - Perbaiki akar penyebab tersebut

S/W Quality Assurance

UNIVERSITAS BAKRIE

Six Sigma dalam Rekayasa Perangkat Lunak

- **Six Sigma** adalah metodologi disiplin yang menggunakan data dan analisis statistik untuk mengukur dan meningkatkan kinerja operasional perusahaan dengan mengidentifikasi dan menghilangkan cacat. Tujuannya adalah untuk mengurangi jumlah cacat (defect) hingga hanya 3,4 per 1 juta kejadian, artinya hampir tanpa kesalahan — sangat tinggi standarnya!

Metodologi DMAIC (untuk perbaikan proses yang ada)

Six Sigma memiliki **5 langkah utama** yang dikenal sebagai **DMAIC**:

Langkah	Penjelasan
D - Define	Tentukan kebutuhan pelanggan, hasil yang diharapkan, dan tujuan proyek.
M - Measure	Ukur performa saat ini, kumpulkan data tentang cacat dan defect.
A - Analyze	Analisis data untuk menemukan akar penyebab utama dari cacat.
I - Improve	Perbaiki proses dengan menghilangkan penyebab utama error.
C - Control	Kontrol proses agar tidak mengulangi kesalahan yang sama di masa depan.

Metodologi DMADV (untuk desain proses baru)

Jika organisasi **membangun proses baru dari awal**, Six Sigma menggunakan pendekatan **DMADV**:

Langkah	Penjelasan
D - Define	Tentukan kebutuhan pelanggan dan tujuan proses.
M - Measure	Ukur dan kumpulkan data awal terkait kebutuhan.
A - Analyze	Analisis untuk mendesain proses yang memenuhi kebutuhan.
D - Design	Desain proses baru yang mencegah kesalahan.
V - Verify	Verifikasi bahwa proses tersebut sesuai dengan tujuan dan bebas dari defect.

S/W Quality Assurance

UNIVERSITAS BAKRIE

Software Reliability (Keandalan Perangkat Lunak)

- **Software reliability (keandalan perangkat lunak)** adalah kemungkinan (probabilitas) bahwa sebuah program akan berjalan tanpa kegagalan dalam lingkungan tertentu selama waktu tertentu. Contoh jika program X memiliki reliabilitas 0.999 selama 8 jam pemrosesan, maka dari 1000 kali eksekusi, program diperkirakan gagal hanya sekali.
- Ukuran Keandalan dan Ketersediaan
  - MTBF (Mean Time Between Failure), mengukur rata-rata waktu antar kegagalan:
    - MTBF=MTTF (Mean Time To Failure)+MTTR (Mean Time To Repair)
      - MTTF: Rata-rata waktu software berjalan sebelum gagal.
      - MTTR: Rata-rata waktu untuk memperbaiki setelah gagal.
  - Contoh SafeHome: Jika sistem alarm gagal setelah 1000 jam (MTTF), dan rata-rata butuh 2 jam untuk memperbaiki (MTTR), maka:
    - MTBF=1000+2=1002
  - FIT (Failures in Time)Menyatakan jumlah kegagalan per 1 miliar jam operasi.1 FIT = 1 kegagalan dalam 1.000.000.000 jam.
    - Gunanya: Cocok untuk komponen yang sangat kritis seperti perangkat lunak dalam sistem medis atau keamanan tinggi.

S/W Quality Assurance

Software Reliability (Keandalan Perangkat Lunak) (2)

- Availability (Ketersediaan)
  - Probabilitas bahwa sistem beroperasi sesuai dengan spesifikasi pada waktu tertentu
  - Availability=$MTTF/(MTTF+MTTR)\times 100\%$
  - Contoh SafeHome:
    - MTTF = 1000 jam, MTTR = 2 jam
    - Availability= $1000/(1000+2)\times 100\%=99.8\%$
    - Makin kecil MTTR, makin tinggi availability-nya. Artinya sistem cepat diperbaiki = makin andal digunakan.
- Pengguna tidak peduli seberapa banyak bug dalam sistem, yang penting adalah apakah sistem gagal atau tidak saat digunakan.

S/W Quality Assurance

UNIVERSITAS BAKRIE

Evaluasi dan Diskusi

8. Apa tujuan utama dari Software Quality Assurance (SQA)?

A) Memastikan perangkat lunak aman untuk digunakan.

B) Menjamin kualitas perangkat lunak melalui proses yang terencana dan sistematis.

C) Menangani hubungan dengan pelanggan.

D) Mengelola anggaran proyek perangkat lunak.

S/W Quality Assurance

UNIVERSITAS BAKRIE

Evaluasi dan Diskusi

9. Framework manajemen kualitas yang bukan digunakan dalam SQA untuk organisasi perangkat lunak adalah:

A) ISO 9000

B) Six Sigma

C) CMMI

D) ISO 27001

S/W Quality Assurance

UNIVERSITAS BAKRIE

Evaluasi dan Diskusi

10. Apa yang harus dilakukan untuk mencapai kualitas perangkat lunak yang tinggi dalam pengembangan perangkat lunak?

A) Mengurangi jumlah tinjauan dan pengujian.

B) Mengabaikan umpan balik pengguna.

C) Menerapkan metode rekayasa perangkat lunak yang benar, pengelolaan proyek yang baik, dan pengendalian kualitas yang ketat.

D) Menghemat biaya dengan mengurangi jumlah pengujian.

Any Questions?

